

3. Grundlagen

Michael Schöttner

Grundlagen Verteilter Systeme

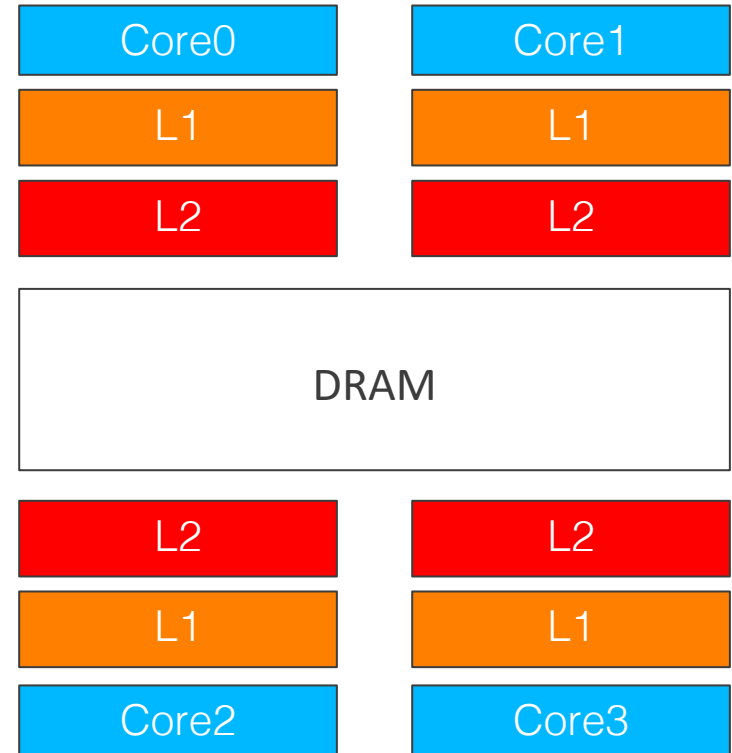
SS 2026

3.1 Vorschau

- Prozesse
- Threads in Java
- Sperren In Java
- Sichtbarkeit von Speicherzugriffen

Speicherhierarchie

- Viele CPUs bestehen aus mehreren oder vielen Kernen (engl. cores)
- Jeder Kern hat eigene Caches
- Zugriffszeiten:
 - L1 Cache: $\sim 1\text{ns}$
 - L2 Cache: 3ns
 - L3 Cache: 15ns
 - DRAM: $\sim 70\text{ns}$
 - SSD: $\sim 150\mu\text{s}$



Begriffe

- **Prozess:** Adressraum für einen oder mehrere Threads
- **Thread:** gehört zu einem Prozess
 - Kann auf alle Daten im Adressraum zugreifen
 - Alle Daten im Heap
 - Offene Dateien & Geräte
 - Hat aber einen eigenen Stack.
- Kooperierende Prozesse können auf einem oder verteilt auf mehreren Rechnern ablaufen

Prozesse: fork () -Beispiel (Auszug)

```
/* process.c */
int main() {
    int pid; /* The pid of a child process. */

    printf("parent: pid=%d\n", (int)getpid());
    pid = fork();

    /* parent code */
    if ( pid ) { /* Value is non-zero in the parent. */
        printf("parent: data=%s\n", data);
    }

    /* child code */
    else {
        printf("    child: pid=%d\n", (int)getpid());
    }

    /* shared code */
    printf("finished\n");
}
```

gcc, Sprachstandard gnu99

Pipes

- Bei einer parallelen Berechnung mithilfe von Kindprozessen müssen diese ihre Ergebnisse an den Eltern-Prozess zurückmelden.
- Aufgrund der getrennten Adressräume geht das nicht direkt, sondern über standardmäßig über Pipes¹
 - Zuverlässige Kommunikation
 - Ordnung der Bytes bleibt erhalten
 - Blockierung bei voller/leerer Pipe
 - Anonyme Pipe mit zwei Handles (read/write)

¹ alternativ via Shared-Memory oder Dateien

Prozesse: Einfaches Pipe-Beispiel

Elternprozess schickt einen String zum Kindprozess

```
int main() {
    char data[80];
    int  rb;                                /* nr of read bytes */
    int  pipe_ends[2];                      /* handles: read=0; write=1 */

    pipe(pipe_ends);                        /* create anonymous pipe */
    if (fork()==0) {                        /* child process */
        close(pipe_ends[1]);                /* close write end -> child wants to read */
        rb = read(pipe_ends[0], data,79);
        data[rb]='\0';                     /* terminate string */
        printf ("Received from parent: %s\n",data);
        close(pipe_ends[0]);                /* done, close read end */
    }
    else {                                  /* parent process */
        close(pipe_ends[0]);                /* close read end -> parent wants to write */
        write(pipe_ends[1],"Hello World!",5);
        close(pipe_ends[1]);                /* done, close write end */
    }
}
```

simple_pipe.c

Bemerkungen

- Parallele Berechnungen werden nicht mehr durch mehrere Prozesse auf einem Computer gelöst, da dies langsam ist
- Gründe:
 - Datenaustausch zwischen Prozessen ist langsam
 - Daten müssen zwischen den Adressräumen hin- und her kopiert werden (zudem auch serialisiert werden, siehe später)
 - Prozesserzeugung ist aufwändig
- Verbleibender Vorteil:
 - Falls die Anwendung später auf ein verteiltes System übertragen werden soll, ist das einfacher, wenn die Anwendung auf parallelen Prozessen basiert

Threads

- Parallelisierung durch Threads muss explizit programmiert werden
- Threads starten bei einer definierten Einstiegsfunktion
- Threads können verschiedene Funktionen eines Programms nebenläufig und/oder simultan ausführen.
- Multicore-CPU's können mehrere Threads **simultan** ausführen, auch von verschiedenen Prozessen (→ separate TLBs pro Core)

Threads: Erzeugung und Terminierung

- Jedes Java-Programm hat einen Haupt-Thread
- Erzeugen von Threads:
 - Interface `Runnable` implementieren (oder Klasse `Thread` erweitern)
 - Methode `run` ist Startpunkt des Threads, wird **nicht** von der Anwendung gerufen, sondern indirekt über `start`
- Terminierung von Threads:
 - JVM/Programm terminiert erst, wenn **alle** Threads terminiert haben¹
 - `tid.join` erlaubt es auf die Terminierung eines bestimmten Threads `tid` blockierend zu warten → Koordination
 - Warten eines Threads ist mit `tid.interrupt` unterbrechbar

¹ C-Programme terminieren, sobald der Haupt-Thread terminiert

Beispiel: Koordination von Threads

```
public class ThreadExample implements Runnable {
    char thName;

    public ThreadExample (char name) { thName = name; }

    public void run () {
        for (int i=0; i<10; i++) {
            System.out.print(thName);
        }
        System.out.println();
    }

    public static void main( String[] args ) {
        Thread t1 = new Thread( new ThreadExample('A') );
        Thread t2 = new Thread( new ThreadExample('B') );
        Thread t3 = new Thread( new ThreadExample('C') );
        try {t1.start(); t1.join();
            t2.start(); t2.join();
            t3.start(); t3.join();
        } catch (InterruptedException e1) { }
        System.out.println("** ENDE **");
    }
}
```

ThreadExample.java

Beispiel: Threads arbeiten auf gemeinsamer Variable

```
public class Incrementer extends Thread {
    static Incrementer lnx, rxz;
    long count = 10;
    static long asyn = 0;

    public static void main( String[ ] s ){
        lnx = new Incrementer();
        rxz = new Incrementer();
        lnx.start(); rxz.start();
    }

    public void run( ){
        long zwisch;
        while ( count-- > 0 ) {
            zwisch = asyn+1;
            asyn = zwisch;
        }
        System.out.println("asyn: " + asyn);
    }
}
```

Incrementer.java

Beispiel: Threads arbeiten auf gemeinsamer Variable

```
public class Incrementer extends Thread {
    static Incrementer lnx, rxz;
    long count = 10000*10000;
    static long asyn = 0;

    public static void main( String[ ] s ){
        lnx = new Incrementer();
        rxz = new Incrementer();
        lnx.start(); rxz.start();
    }

    public void run( ){
        long zwisch;
        while ( count-- > 0 ) {
            zwisch = asyn+1;
            asyn = zwisch;
        }
        System.out.println("asyn: " + asyn);
    }
}
```

Falls genau hier die CPU/Core entzogen wird,
kann es zu einem **Lost-Update** kommen

Incrementer.java

Threads: Wettlaufsituation

- Sobald mehrere Threads in einem Prozessadressraum ausführt werden kann es zu Wettlaufsituationen kommen
- **Wettlaufsituation** (engl. race condition): wenn nebenläufige Threads eines Prozesses auf gemeinsame Variablen schreiben, so ist das Ergebnis nicht deterministisch.
- **Lost update**: zum Beispiel wenn zwei Threads gleichzeitig auf eine gemeinsame Variable schreibend zugreifen

Bemerkungen

- Parallele Berechnungen auf einem Computer sind viel effizienter wenn diese mithilfe von mehreren Threads realisiert werden
- Gründe:
 - Threads können auf alle Daten im gleichen Prozess direkt zugreifen
 - Threaderzeugung geht schneller als Prozesserzeugung
- Nachteil:
 - Falls die Anwendung später auf ein verteiltes System übertragen werden soll, ist dies deutlich schwieriger
- Bei großen verteilten parallelen Anwendungen nutzt man beides:
 - Meist ein Prozess der Anwendung auf jedem beteiligten Computer
 - Und jeder dieser Prozesse führt mehrere Threads aus (mindestens ein Thread pro Core)

3.4 Synchronisierung

- Wettlaufsituationen und Lost-Updates müssen mithilfe von Synchronisierung vermieden werden
- Hierzu verwendet man **kritische Abschnitte** (engl. critical regions)
 - = Programmabschnitt in dem sich nur ein Thread befinden
 - Ein- und Austritt in den kritischen Abschnitt wird synchronisiert
- Java bietet hierzu eine Reihe von **Sperren**, z.B. `synchronized`
 - Es gibt eine Sperre pro Objekt – in Java ist (fast) alles ein Objekt¹
 - Als Methodenattribut → Instanz oder Klassenobjekt (`static`) wird gesperrt
 - Oder angewandt auf eine Referenz: `synchronized(ref)` ;
→ sperrt die durch `ref` referenzierte Instanz

¹ ausgenommen primitive Datentypen

Beispiel: Lost Update mit Synchronisierung

```
public class IncrementerSynced extends Thread {
    static Object mutex = new Object();
    static IncrementerSynced lnx, rxz;
    long count = 10000*10000;
    static long syn = 0;

    public static void main( String[ ] s ){
        lnx = new IncrementerSynced();
        rxz = new IncrementerSynced();
        lnx.start(); rxz.start();
    }

    public void run( ){
        long zwisch;
        while ( count-- > 0 ) {
            synchronized (mutex) {
                zwisch = syn+1;
                syn = zwisch;
            }
        }
        System.out.println("syn: " + syn);
    }
}
```

CPU/Core-Entzug ist an dieser Stelle immer noch möglich, aber jetzt kein Problem mehr.

IncrementerSynced.java

Beispiel: Lost Update mit Synchronisierung

```
public class IncrementerSynced extends Thread {
```

```
    static Object mutex = new Object();
```

```
    static IncrementerSynced lnx, rxz;
```

```
    long count = 10000*10000;
```

```
    static long syn = 0;
```

```
    public static void main( String[ ] s ){
```

```
        lnx = new IncrementerSynced();
```

```
        rxz = new IncrementerSynced();
```

```
        lnx.start(); rxz.start();
```

```
    }
```

```
    public void run( ){
```

```
        long zwisch;
```

```
        while ( count-- > 0 ) {
```

```
            synchronized (mutex) {
```

```
                zwisch = syn+1;
```

```
                syn = zwisch;
```

```
            }
```

```
        }
```

```
        System.out.println("syn: " + syn);
```

```
    }}
```

← Was passiert wenn wir static weglassen?

IncrementerSynced.java

Signalisierung bei `synchronized`

- Manchmal muss man nach dem Belegen einer Sperre auf ein Ereignis warten
- Problematisch ist, wenn zum Auslösen dieses Ereignisses ebenfalls die gleiche Sperre erworben werden muss.
- Ohne weitere Synchronisierungskonzepte ist das Problem nicht lösbar
- Hierfür bietet Java, als auch andere Programmiersprachen, ein Signalisierungskonzept → `wait` und `notify`

Signalisierung bei `synchronized`

- `obj.wait`
 - Aufrufer gibt Sperre auf `obj` implizit frei und wird blockiert
 - Nur erlaubt, wenn eine Sperre auf `obj` gehalten wird
 - Optional auch mit Timeout
- `obj.notify`
 - Deblockiert einen Thread, der zuvor `obj.wait` gerufen hat
 - Letzterer läuft erst weiter, wenn der Aufrufer von `obj.notify` die Sperre auf `obj` freigegeben hat
 - Wartet kein Thread, so wird das Ereignis „notify“ verworfen. Es wird also nicht zwischengespeichert für später.

Beispiel: Implementierung von Semaphoren

```
public class Semaphore {  
    private int count;  
  
    public synchronized void P() throws InterruptedException {  
        while (count <= 0) wait();  
        count--;  
    }  
  
    public synchronized void V() {  
        count++;  
        notify();  
    }  
  
    public Semaphore(int capacity) {  
        count = capacity;  
    }  
}
```

Falls kein Thread darauf wartet,
wird das Ereignis verworfen

Semaphore.java

- Gibt es auch fertig in `java.util.concurrent.Semaphore`

Beispiel: Anwendung der Semaphore

```
public class IncrementerSema extends Thread {
    static Semaphore sema = new Semaphore(1); // we use our own Semaphore
    static IncrementerSema lnx, rxz;
    long count = 10000*10000;
    static long syn = 0;

    public static void main( String[ ] s ){
        lnx = new IncrementerSema();
        rxz = new IncrementerSema();
        lnx.start(); rxz.start();
    }

    public void run( ){
        long zwisch;
        while ( count-- > 0 ) {
            sema.P();
            zwisch = syn+1;
            syn = zwisch;
            sema.V();
        }
        System.out.println("syn: " + syn);
    }
}
```

IncrementerSema.java

Beispiel: Anwendung der Semaphore

- Was ist der Vorteil von **synchronized** gegenüber unserer Semaphore?

Beispiel: Anwendung der Semaphore

- Was ist der Vorteil von **synchronized** gegenüber unserer Semaphore?
- Bei **synchronized** wird die Sperre automatisch freigegeben, wenn der Scope endet.

ReentrantReadWriteLock

- Ein wechselseitiger Ausschluss ist sehr restriktiv → u. U. werden Threads auf allen anderen Cores durch eine Sperre aufgehalten
- Häufig sollen mehrere lesende Threads parallel arbeiten dürfen oder nur einen Schreiber
- **Lösung: ReentrantReadWriteLock**
 - Besteht intern aus zwei Locks
 - `readLock().lock()` blockiert, eine Schreibe-Sperre vergeben ist und wartet auf Freigabe; mehrere Leser-Sperren sind erlaubt
 - `writeLock().lock()` blockiert, wenn eine Lese- oder eine Schreib-Sperre vergeben ist und wartet auf Freigabe
 - Optionale Fairness einstellbar über Konstruktor

Beispiel

- 4 Threads die auf einem gemeinsamen String zugreifen
- 2 Threads geben den String auf der Konsole aus → Reader
- 2 Threads hängen konkurrierend einen Buchstaben an das String-Ende an
→ Thread 1 immer „a“ und Thread 2 immer „b“.
→ Writer
- Alle 4 Threads führen jeweils eine Schleife aus,
wobei jeder Schleifendurchlauf synchronisiert wird.
- Der Zugriff auf den String (Klassenvariable) wird über eine Lese- und eine Schreib-Sperre synchronisiert.

Beispiel: Code-Auszug

```
import java.util.concurrent.locks.ReentrantReadWriteLock;
```

```
public class ReentrantReadWriteLockExample {
```

```
    private static int NUM_INTERACTIONS = 10;
```

```
    private static String message = "";
```

```
    private static ReentrantReadWriteLock lock = new ReentrantReadWriteLock(true);
```

```
    static class Read implements Runnable {
```

```
        public void run() {
```

```
            for (int i=0; i<=NUM_INTERACTIONS; i++) {
```

```
                if (lock.isWriteLocked()) {
```

```
                    System.out.println("I'll take the lock from Write");
```

```
                }
```

```
                try {
```

```
                    lock.readLock().lock();
```

```
                    System.out.println(" ---> Message is " + message );
```

```
                } finally {
```

```
                    lock.readLock().unlock();
```

```
                }
```

```
            }  
        }  
    }  
}
```

Fairness



Globale Variable



Benötigt keine Instanz
von Outer-Class



Always call unlock



Lesezugriff
auf die
globale
Variable



ReentrantReadWriteLockExample.java

Beispiel: Code-Auszug

```
static class Writer implements Runnable {  
    String ch;  
  
    public Writer(String ch) {  
        this.ch = ch;  
    }  
  
    public void run() {  
        for(int i = 0; i <= NUM_ITERATIONS; i++) {  
            try {  
                lock.writeLock().lock();  
                message = message.concat(ch);  
            } finally {  
                lock.writeLock().unlock();  
            }  
        }  
    }  
}
```

Always call unlock

Schreibzugriff auf
die globale Variable

ReentrantReadWriteLockExample.java

Bemerkungen zum Code-Beispiel

- Leser können gleichzeitig parallel laufen
- Schreiber benötigen exklusiven Zugriff
- Bei aktivierter Fairness werden wartende Schreiber nicht unbegrenzt durch Leser verhindert

Was ist besser? Fair oder unfair?

Was ist besser? Fair oder unfair?

- Hängt von der Anwendung ab
- Fair: Zugriffe werden ungefähr in Reihenfolge des Wartens vergeben.
 - Gerechter → verhindert Verhungern
 - Langsamer durch Overhead
- Unfair: Wartezeit von Threads wird nicht berücksichtigt
 - höherer Durchsatz → weniger Threadwechsel
 - aber evt. verhungern einzelne Threads

3.5 Java-Speichermodell

- Das Memory-Modell von Java regelt drei Dinge
- **Atomicity**: Welche Operationen sind atomar, d.h. werden nicht durch andere Threads unterbrochen?
- **Ordering**: In welcher Reihenfolge sehen Threads Schreibzugriffe? (Speichergriffe können zwischen verschiedenen Threads umsortiert werden)
- **Visibility**: Wann werden Modifikationen eines Threads den anderen Threads sichtbar gemacht?
- Siehe unter anderem: <http://www.angelikalanger.com/Articles/PDF/Java-Memory-Model.pdf>
- Memory-Modell auf Hardware-Ebene regelt wann Schreibzugriffe zwischen Cores und deren Caches sichtbar werden

Beispiel: Visibility.java

- Terminiert das Programm?

```
public class Visibility extends Thread {  
  
    boolean keepRunning = true;  
  
    public static void main(String[] args) throws InterruptedException {  
        Visibility t = new Visibility();  
        t.start();  
        Thread.sleep(1000);  
        t.keepRunning = false;  
        System.out.println("Main-Thread: keepRunning is now false");  
    }  
  
    public void run() {  
        while (keepRunning) {  
            ;  
        }  
        System.out.println("Second-Thread: finished");  
    }  
}
```

Visibility.java

Visibility im Java-Speichermodell

- = regelt wann Änderungen eines Threads für andere Threads sichtbar werden
 - **Refresh**: Schreibzugriffe anderer Threads werden gelesen
 - **Flush**: Schreibzugriffe werden anderen Threads sichtbar gemacht
- Threadstart und -ende:
 - Start: löst Refresh aus
 - Ende: löst einen Flush aus
 - nach einem `join` sehen wartende Threads alle Änderungen
- Synchronisierung (`synchronized` & andere Locks)
 - Erhalt einer Sperre → Refresh
 - Freigabe einer Sperre → Flush
 - Zwei Threads die sich über die gleiche Sperre synchronisieren sehen entsprechend gegenseitig alle Schreibzugriffe

Visibility im Java-Speichermodell

- Volatile Variablen
 - Lesen → Refresh
 - Schreiben → Flush
 - Erzwingt den Variablenzugriff immer über Hauptspeicher (nicht Cache)
 - Achtung: volatile Referenzen beschränken Verhalten auf Referenzen und erfassen nicht die Variablen der Instanz

Beispiel: Visibility.java

- Terminiert das Programm?

```
public class Visibility extends Thread {  
    volatile boolean keepRunning = true; ← volatile löst das Problem  
  
    public static void main(String[] args) throws InterruptedException {  
        Visibility t = new Visibility();  
        t.start();  
        Thread.sleep(1000);  
        t.keepRunning = false;  
        System.out.println("Main-Thread: keepRunning is now false");  
    }  
  
    public void run() {  
        while (keepRunning) {  
            ;  
        }  
        System.out.println("Second-Thread: finished");  
    }  
}
```

Visibility.java

Beispiel: Visibility.java

- Terminiert diese Version?

```
public class Visibility extends Thread {  
  
    boolean keepRunning = true;  
  
    public static void main(String[] args) throws InterruptedException {  
        Visibility t = new Visibility();  
        t.start();  
        Thread.sleep(1000);  
        t.keepRunning = false;  
        System.out.println("Main-Thread: keepRunning is now false");  
    }  
  
    public void run() {  
        while (keepRunning) {  
            System.out.println("uncomment to solve visibility issue");  
        }  
        System.out.println("Second-Thread: finished");  
    }  
}
```

Ausgaben in Standard-Ouptut werden synchronisiert.

Visibility.java

Zusammenfassung

- Nebenläufigkeit und simultane Ausführung mehrere Threads ist in verteilten Anwendungen der Standardfall (sowohl lokal als auch global)
- Synchronisierung zwischen Threads muss sowohl lokal als auch global durchdacht werden
- Wie und wann Änderungen im lokalen Speicher für andere beteiligte Computer sichtbar werden stellt sich global noch offensichtlicher wie lokal